
Modül 19 — Native Image ve Performans

Ders Notları

Spring Boot Edu



2026-09-25

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Başlangıç Süresi Neden Önemli	2
2.2 Başlangıçta Ne Olur ve Ne Build Zamanına Taşınabilir	2
2.3 Kapalı Dünya Varsayımı	3
3. Adım Adım Örnekler	3
3.1 Uygulama	3
3.2 AOT İşleme ve AOT Repository'leri	4
3.3 AOT'un Yetmediği Yer: Dinamik Kod	6
3.4 RuntimeHints	6
3.5 Native Image'ı Build Etmek	7
3.6 JVM AOT Cache ve Karşılaştırma	8
4. Sık Yapılan Hatalar ve En İyi Pratikler	9
5. Özet	10
6. İleri Okuma	10

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Spring'in AOT işleminin build sırasında ne yaptığını açıklamak ve üretilen kodu okumak
- Spring Data AOT repository'lerini kullanmak ve AOT işleminin veritabanı olmadan neden biraz yapılandırmaya ihtiyaç duyduğunu bilmek
- Reflection ve kaynaklar için RuntimeHints yazmak ve onları test etmek
- GraalVM kurmadan Paketo buildpacks ile bir GraalVM native image build etmek
- Normal bir JVM başlangıcını JDK'nın AOT cache'i (Project Leyden) ile hızlandırmak
- Başlangıç süresini ve belleği ölçmek ve bir iş yükü için doğru varyantı seçmek

Ön koşullar: Modül 03 (Web MVC), Modül 05 (Spring Data JDBC) · **Tahmini süre:** 4 saat
· **Docker gerekir**

2. Kavramlar

2.1 Başlangıç Süresi Neden Önemli

Haftada bir başlayan, uzun süre çalışan bir sunucu 3 saniyeyi umursamaz. Ama bazı ortamlarda başlangıç kritik yol üzerindedir: yük altında ölçeklenme, sıfıra ölçeklenme (serverless), kısa batch işleri, CLI araçları ve geliştirmede hızlı geri bildirim. Birçok örnek yan yana çalıştığında bellek önemlidir.

2.2 Başlangıçta Ne Olur ve Ne Build Zamanına Taşınabilir

Normal bir başlangıçta Spring yapılandırma sınıflarını okur, koşulları değerlendirir, bean tanımlarını reflection ile oluşturur ve Spring Data metot adlarından sorgular türetir. Bunların hepsi her başlangıçta aynıdır.

Teknik	Build zamanına taşınan	Çalıştığı yer
Spring AOT (process -aot)	üretilmiş Java kodu olarak bean tanımları, repository gerçekleştirimleri, ipuçları	JVM veya native
JVM AOT cache (JEP 483, 514, 515)	bir eğitim çalıştırmasından yüklenmiş ve bağlanmış sınıflar, metot profilleri	JVM (JDK 25+)

Teknik	Build zamanına taşınan	Çalıştığı yer
GraalVM native image	makine koduna derlenmiş uygulamanın tamamı (kapalı dünya)	native çalıştırılabilir dosya

2.3 Kapalı Dünya Varsayımı

native-image uygulamayı build sırasında analiz eder ve yalnızca ulaşabildiğini dahil eder. Dinamik olarak ulaşılan kod (sınıf adıyla reflection, kaynaklar, proxy'ler, serileştirme) bu analize görünmezdir. **Reachability metadata** olarak bildirilmelidir. Spring'in AOT motoru bu metadata'nın çoğunu kendisi yazar (bean'ler, controller'lar, controller metodlarının Jackson tipleri için). Kendi dinamik kodunuz için RuntimeHints yazarsınız.

i Not

SPEC karar 8: Henüz JDK 27 için GraalVM yok. Bu modül `maven.compiler.release=25` ile derlenir: Aynı sınıflar JDK 27'de çalışır (JVM, AOT cache) ve GraalVM 25 onları bir native image'a dönüştürür.

3. Adım Adım Örnekler

Docker çalışırken uygulamayı JVM'de başlatın. PostgreSQL kök `compose.yaml`'dan başlar:

```
./mvnw -pl modules/19-native-performance/lesson spring-boot:run
```

Tur, uygulamanın nasıl çalıştığını yazdırır:

```
=== mode: JVM, ready in 1402 ms ===
Designing Data-Intensive Applications – ₺110,00
Effective Java – ₺89,90
Spring in Action – ₺95,00
```

3.1 Uygulama

Küçük bir kitapçı API'si: GET `/api/books?title=...`, GET `/api/books/{isbn}`, GET `/api/books/{isbn}/price`, GET `/api/books/since/{year}` ve GET `/api/quote` (bkz. requests.http). Aynı jar JVM'de, AOT cache ile ve native image olarak ölçülür.

Tur modu Spring'den okur:

```
@EventListener
void ready(ApplicationReadyEvent event) {
    String mode = NativeDetector.inNativeImage() ? "native image"
        : AotDetector.useGeneratedArtifacts() ? "JVM with Spring AOT" : "JVM";
    Duration startup = event.getTimeTaken();
    System.out.println("=== mode: " + mode + ", ready in " + startup.toMillis() + " ms
        ↪ ===");
    books.publishedSince(2017).forEach(book ->
        System.out.println(" " + book.title() + " - " + prices.format(book.price())));
}
```

3.2 AOT İşleme ve AOT Repository'leri

Ders, process-aot'u yalnızca native profiline değil her build'e bağlar:

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <!-- AOT processing in every build (not only with -Pnative): the jar can then
        ↪ also run
           on the JVM with -Dspring.aot.enabled=true, and AotProcessingIT checks the
        ↪ output -->
    <execution>
      <id>process-aot</id>
      <goals>
        <goal>process-aot</goal>
      </goals>
    </execution>
  </executions>
  <configuration>
    <image>
      <name>springbootedu/native-performance:${project.version}</name>
      <env>
        <BP_JVM_VERSION>25</BP_JVM_VERSION>    <!-- GraalVM / Liberica NIK 25 -->
      </env>
    </image>
  </configuration>
</plugin>
```

./mvnw -pl modules/19-native-performance/lesson package sonrasında lesson/target/spring-aot/main/ içine bakın:

```

sources/com/springbootedu/nativeperformance/
  NativePerformanceApplication__BeanFactoryRegistrations.java    all bean definitions, as
↪ code
  book/BookController__BeanDefinitions.java                    "new BookController(...)"
↪ instead of reflection
  book/BookRepositoryImpl__AotRepository.java                  the repository's queries,
↪ as code
resources/META-INF/native-image/com.springbootedu/19-native-performance-lesson/
  reachability-metadata.json                                    reflection and resource
↪ metadata for GraalVM

```

Repository arayüzü her zamanki gibi kalır:

```

public interface BookRepository extends ListCrudRepository<Book, String> {

    List<Book> findByTitleContainingIgnoreCaseOrderByTitle(String part);

    @Query("SELECT * FROM book WHERE year >= :year ORDER BY title")
    List<Book> publishedSince(int year);
}

```

AOT ile Spring Data, `findByTitleContainingIgnoreCaseOrderByTitle`'in SQL'ini başlangıçta türetmez. `BookRepositoryImpl__AotRepository` onu zaten içerir. Bu SQL'i build sırasında yazmak için Spring Data'nın veritabanı dialect'ine ihtiyacı vardır. Normalde Boot veritabanına sorar, ama build sırasında veritabanı yoktur:

```

@Configuration(proxyBeanMethods = false)
class JdbcDialectConfiguration {

    @Bean
    JdbcPostgresDialect jdbcDialect() {
        return JdbcPostgresDialect.INSTANCE;
    }
}

```

Bu bean olmadan `process-aot`, `Failed to determine a suitable driver class` ile başarısız olur: Dialect'i bulmak için `DataSource`'u oluşturmaya çalışır.

! Önemli

`process-aot` uygulama context'ini build sırasında başlatır (yalnızca bean tanımları, bean'leri başlatmadan). Hangi bean'lerin var olduğuna karar veren her şey, örneğin profiller ve `@ConditionalOnProperty`, build sırasında sabitlenir. Çalışma zamanında açtığınız bir profil, AOT ile işlenmiş bir uygulamaya bean ekleyemez.

AotProcessingIT, package'tan sonra çalışır ve üretilen dosyaları kontrol eder. Jar'ı JVM'de üretilen kodla çalıştırmak için `-Dspring.aot.enabled=true` ekleyin.

3.3 AOT'un Yetmediği Yer: Dinamik Kod

Uygulamanın iki parçası bilerek dinamiktir. Fiyat formatı sınıf adıyla yapılandırılır (`bookstore.price-format.class-name`) ve reflection ile oluşturulur:

```
private static PriceFormat instantiate(String className) {
    try {
        Class<?> type = Class.forName(className); // invisible for
        ↪ static analysis
        return (PriceFormat) type.getDeclaredConstructor().newInstance();
    } catch (ReflectiveOperationException e) {
        throw new IllegalStateException("Cannot create price format " + className, e);
    }
}
```

QuoteOfTheDay, classpath'ten `quotes/quotes.txt`'yi okur. JVM'de ikisi de her zaman çalışır. Bir native image'da `TurkishLiraFormat` kaldırılırdı (hiçbir kod constructor'ını çağırıyor) ve `quotes.txt` image'da olmazdı.

3.4 RuntimeHints

```
public class BookstoreRuntimeHints implements RuntimeHintsRegistrar {

    @Override
    public void registerHints(RuntimeHints hints, @Nullable ClassLoader classLoader) {
        // PriceFormats creates these by name: keep the classes and allow calling their
        ↪ constructors
        hints.reflection()
            .registerType(TurkishLiraFormat.class,
                ↪ MemberCategory.INVOKE_DECLARED_CONSTRUCTORS)
            .registerType(EuroFormat.class,
                ↪ MemberCategory.INVOKE_DECLARED_CONSTRUCTORS);
        // QuoteOfTheDay reads this file: include it in the image
        hints.resources().registerPattern("quotes/*.txt");
    }

    @Configuration(proxyBeanMethods = false)
    @ImportRuntimeHints(BookstoreRuntimeHints.class)
    static class Registration {
    }
}
```

İpuçları düz veridir, bu yüzden bir birim testi onları image build etmeden kontrol edebilir:

```
@Test
void theHintsCoverTheReflectionAndTheResources() {
    RuntimeHints hints = new RuntimeHints();
    new BookstoreRuntimeHints().registerHints(hints, getClass().getClassLoader());

    assertT-
    ↪ hat(RuntimeHintsPredicates.reflection().onType(TurkishLiraFormat.class)).accepts(hints);
    assertT-
    ↪ hat(RuntimeHintsPredicates.reflection().onType(EuroFormat.class)).accepts(hints);
    assertT-
    ↪ hat(RuntimeHintsPredicates.resource().forResource("quotes/quotes.txt")).accepts(hints);
}
```

İpucu vermenin diğer yolları:

Anotasyon	Ne için
@ImportRuntimeHints(MyHints.class)	bir RuntimeHintsRegistrar kaydeder (yukarıdaki gibi)
@RegisterReflectionForBinding(Dto.class)	kendiniz (de)serialize ettiğiniz tipler, örneğin JsonSerializer ile
@Reflective	reflection ile çağrılan bir metot veya sınıf

★ İpucu

Bir native image ClassNotFoundException, NoSuchMethodException veya eksik bir kaynakla başarısız olursa bir ipucuna ihtiyacınız vardır. GraalVM tracing agent'ı (-agentlib:native-image-agent), uygulama JVM'de çalışırken metadata'yı bir başlangıç noktası olarak kaydedebilir.

3.5 Native Image'ı Build Etmek

Spring Boot'un native profili process-aot'u çalıştırır ve jar'ı hazırlar. Ardından Paketo buildpacks, GraalVM'i (BellSoft Liberica NIK 25) Docker içinde çalıştırır:

```
./mvnw -Pnative -pl modules/19-native-performance/lesson spring-boot:build-image
```

Image adı ve buildpack için Java sürümü yukarıdaki process-aot snippet'inde ayarlanır (BP_JVM_VERSION=25). İlk build builder'ı ve GraalVM'i indirir ve birkaç dakika sürer. native-image, Docker içinde yaklaşık 4 GB boş belleğe ihtiyaç duyar.

! Uyarı

Build The Native Image build process ran out of memory (exit status 137) ile durursa Docker'ın belleğini başka container'lar kullanıyordur. İhtiyacınız olmayan servisleri durdurun, örneğin docker compose --profile all stop elasticsearch kafka lgdm mongo hazelcast redis, veya Docker'a daha fazla bellek verin.

Native image'ı compose PostgreSQL'ine karşı başlatın:

```
docker run --rm -p 8080:8080 springbootedu/native-performance:1.0.0-SNAPSHOT \
  --spring.datasource.url=jdbc:postgresql://host.docker.internal:5432/bookstore \
  --spring.datasource.username=bookstore --spring.datasource.password=bookstore
```

```
=== mode: native image, ready in 173 ms ===
Designing Data-Intensive Applications – ₺110,00
```

Bu dersin ilk native build'i ₺110,00 yerine ₺110.00 yazdırdı: Reflection ipucu çalıştı, ama bir native image yalnızca varsayılan locale'i (en) içerir. Türkçe ve Almanca sayı formatlarının locale'lere build sırasında ihtiyacı vardır. Uygulamaya ait build seçenekleri META-INF/native-image/ içindeki bir native-image.properties dosyasına konur, böylece build-packs, native:compile ve CI için aynı şekilde çalışır:

```
Args = -H:IncludeLocales=tr-TR,de-DE
```

Yerel bir GraalVM 25 (GRAALVM_HOME) ve JDK 27 üzerinde Maven ile ./mvnw -Pnative -pl modules/19-native-performance/lesson native:compile kendi işletim sisteminiz için bir çalıştırılabilir dosya build eder. CI workflow'u native.yml tam olarak bunu Linux'ta yapar ve çalıştırılabilir dosyayı smoke test'ten geçirir. Birkaç dakika sürdüğü için elle başlatılır.

3.6 JVM AOT Cache ve Karşılaştırma

JDK 24'ten beri (JEP 483) JVM, yüklenmiş ve bağlanmış sınıfları bir **AOT cache**'te saklayabilir. JDK 25 bunu tek adımlık bir komut yaptı (JEP 514) ve metod profillerini ekledi (JEP 515). Bir **eğitim çalıştırması** uygulamayı başlatır ve cache'i yazar. Spring Boot -Dspring.context.exit=onRefresh ile yardım eder: Uygulama context hazır olur olmaz durur.

```
java -XX:AOTCacheOutput=app.aot -Dspring.aot.enabled=true
↪ -Dspring.context.exit=onRefresh -jar app.jar
java -XX:AOTCache=app.aot -Dspring.aot.enabled=true -jar app.jar
```

Cache yalnızca aynı JDK ve aynı classpath ile çalışır. Bu yüzden Boot açılmış (extracted) bir jar önerir: java -Djarmode=tools -jar app.jar extract --destination application.

`compare-startup.sh` bunların hepsini yapar ve her varyantı ölçer (birkaç çalıştırmanın medyanı):

```
modules/19-native-performance/compare-startup.sh 5
```

Varyant (5 çalıştırmanın medyanı)	Başlangıç ms	RSS MB
JVM	1716	296
JVM + Spring AOT	1623	267
JVM + Spring AOT + AOT cache	736	208
Native image (GraalVM 25)	226	71

- **Tek başına Spring AOT** burada az kazandırır: Küçük bir uygulamanın context'i zaten hızlı kurulur. Çok sayıda bean ile daha önemlidir ve bir native image'ın ön koşuludur.
- **AOT cache** başlangıcı yarıdan fazla kısaltır: JVM başlangıcının en büyük kısmı olan sınıf yükleme ve bağlama eğitim çalıştırmasında yapılır.
- **Native image**, düz JVM'den yaklaşık 7 kat hızlı başlar ve belleğin yaklaşık dörtte birine ihtiyaç duyar.

“Başlangıç”, Boot'un “process running for” değeridir, bu yüzden JVM'in kendi başlangıcını içerir. Native bellek `docker stats` ile, JVM belleği süreç RSS'i olarak ölçülür. Sayılar bir MacBook'tan (Apple silicon) alınmıştır ve sizin makinenizde farklıdır: milisaniyeleri değil oranları karşılaştırın.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Build edilen bir native image, çalışan bir native image değildir. Dinamik kod yalnızca çalıştığında başarısız olur. Native çalıştırılabilir dosyayı test edin (CI smoke test'i reflection ve kaynak kullanan endpoint'leri çağırır) veya testleri `-PnativeTest` ile native çalıştırın.

- **Yapın:** `RuntimeHints`'inizi her kod gibi `RuntimeHintsPredicates` ile test edin.
- **Yapmayın:** Çalışma zamanı profillerinin veya `@ConditionalOnProperty`'nin AOT ile işlenmiş bir uygulamanın bean'lerini değiştirmesini beklemeyin. Onlara build sırasında karar verin.
- **Yapın:** Öncesini ve sonrasını ölçün. AOT cache neredeyse bedavadır (bir eğitim çalıştırması ve bir JVM bayrağı). Bir native image build'i, hata ayıklamayı ve kullanabileceğiniz kütüphaneleri değiştirir.

- **Yapmayın:** Bir native image'ı bir JVM ile yalnızca başlangıç süresine göre karşılaştırmayın. Isındıktan sonra JVM'in JIT derleyicisi çoğu zaman bir native image'dan daha yüksek bir verim (throughput) elde eder.
- **Yapın:** Eğitim çalıştırmasını gerçek kullanıma yakın tutun, böylece cache production'ın ihtiyaç duyduğu sınıfları içerir.

5. Özet

- Spring AOT işi başlangıçtan build zamanına taşır: Bean tanımları ve repository gerçekleştirmeleri üretilmiş kod olur ve ipuçları GraalVM metadata'sı olur.
- AOT işleme veritabanı olmadan çalışır, bu yüzden JDBC dialect'i gibi ayarlar açıkça belirtilmelidir.
- Dinamik olarak ulaşılan kod RuntimeHints'e ihtiyaç duyar (reflection, kaynaklar, binding) ve bu ipuçları birim testlerle test edilebilir.
- Paketo buildpacks Docker içinde bir GraalVM 25 native image build eder. Bir native image milisaniyeler içinde başlar ve az bellek kullanır, ama build hızından ve biraz dinamiklikten vazgeçer.
- JVM AOT cache (Project Leyden), tek bir eğitim çalıştırmasıyla normal bir JVM başlangıcını çok daha hızlı yapar.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Boot — GraalVM Native Image Support](#)
- [Spring Boot — AOT Cache](#)
- [Spring Framework — Ahead of Time Optimizations](#)
- [Spring Data JDBC — AOT Repositories](#)
- [GraalVM — Reachability Metadata](#)
- [JEP 483: Ahead-of-Time Class Loading & Linking](#) · [JEP 514](#) · [JEP 515](#)